

ネットワーク基礎ノート

物理層からプロトコルまで：Ethernet / TCP-IP / CAN

大学講義レジュメ（解説つき）スタイル

2026 年 6 月 21 日

目次

第 1 章	ガイダンス：ネットワークを「層」で考える	5
1.1	なぜ層に分けるのか	5
1.2	OSI 参照モデルと TCP/IP モデル	6
1.3	カプセル化：データが旅に出る前の準備	6
第 2 章	Ethernet	9
2.1	Ethernet の全体像	9
2.2	物理層：ケーブルと速度の規格	9
2.2.1	オートネゴシエーションと全二重/半二重	10
2.3	MAC アドレス：誰宛てかを表す 48 ビット	10
2.4	Ethernet フレームの構造	10
2.4.1	ジャンボフレームと VLAN タグ	11
2.5	かつての主演：CSMA/CD	11
2.6	ハブとスイッチ：衝突ドメインとブロードキャストドメイン	12
2.6.1	スイッチの MAC 学習（要点）	12
第 3 章	TCP/IP	15
3.1	IP：エンドツーエンドの住所と配送	15
3.1.1	IPv4 アドレスとサブネット	15
3.1.2	IPv4 ヘッダ	16
3.1.3	IPv6 のひとつ	16
3.2	ARP と ICMP：IP を支える縁の下	17
3.3	TCP：信頼性のある通信路	17
3.3.1	TCP ヘッダ	17
3.3.2	3 ウェイハンドシェイク（接続確立）	18
3.3.3	信頼性のしくみ（要点）	19
3.4	UDP：軽量・コネクションレス	19
3.5	ポート番号と NAT	19
第 4 章	CAN（Controller Area Network）	21
4.1	CAN の設計思想	21
4.2	物理層：差動信号とドミナント/レセプティブ	21
4.3	CAN フレームの構造（標準フォーマット）	22

4.4	アービトレーション：衝突しても壊れない調停	23
4.5	ビットスタッフィングと同期	23
4.5.1	ビットタイミング（参考）	24
4.6	エラー検出とフォールトコンフィニメント	24
4.6.1	エラーカウンタとバスオフ	25
4.7	CAN FD：データを増やし高速化	25
4.8	Ethernet との棲み分け（まとめにかえて）	25
第 5 章	演習：パケットを解読する	27
5.1	問題	27
5.1.1	与えられた仕様（プロトコル定義）	27
5.1.2	設問	28
5.2	解答・解説	30
5.2.1	設問 1・2：各フィールドの値	30
5.2.2	設問 3：length の整合性	31
5.2.3	設問 4：status_flags のビット展開	31
5.2.4	設問 5：発展（プラス）	31
付録 A	手計算リファレンス：16 進数の読み方	33
A.1	下準備：16 進 1 桁の対応表	33
A.2	16 進数 → 10 進数	33
A.3	複数バイトの並べ替え：エンディアン	34
A.4	unsigned int（符号なし整数）	35
A.5	signed int（符号付き整数・2 の補数）	35
A.6	bool（真偽値）	36
A.7	ビットフラグの展開	36

第1章 ガイダンス：ネットワークを「層」で考える

このノートは、Ethernet・TCP/IP・CAN を「あとから読み返して分かる」ことを目標にまとめた、講義レジュメ風の解説である。各章は独立して読めるが、共通する見方は **階層化（レイヤリング）** なので、最初にそこを押さえておく。



図 1.1 ネットワークは役割ごとに積み重なった「層」でできている。各層は下の層のサービスを使い、上の層へサービスを渡す。

1.1 なぜ層に分けるのか

ネットワークは「ケーブルに電気を流す」ところから「アプリ同士がデータをやり取りする」ところまで、関心事がまったく異なる処理の積み重ねでできている。これを一枚岩で作ると、配線を変えただけでアプリまで作り直しになる。そこで**役割ごとに層を分け、層と層の間の約束（インタフェース）だけを決めておく**。すると、ある層の中身を差し替えても他の層に影響しない。

要点

各層は「下の層が提供するサービスを使い、上の層にサービスを提供する」。隣り合う層の間でやり取りする約束だけが固定で、層の**中身**は自由。これが、有線を無線に変えても

TCP/IP がそのまま動く理由である。

1.2 OSI 参照モデルと TCP/IP モデル

代表的なモデルが 2 つある。理論を整理する 7 層の **OSI 参照モデル** と、実装に即した 4 層の **TCP/IP モデル** である。対応関係を押さえておくと、「これは何層の話か」で迷わなくなる。

OSI 7 層	TCP/IP モデル	本ノートでの主な対象
7 アプリケーション		(HTTP, DNS など)
6 プレゼンテーション	アプリケーション	
5 セッション		
4 トランスポート	トランスポート	TCP / UDP (第 3 章)
3 ネットワーク	インターネット	IP / ARP / ICMP (第 3 章)
2 データリンク		Ethernet MAC / CAN (第 2,4 章)
1 物理	ネットワーク インタフェース	Ethernet PHY / CAN 物理層

表 1.1 OSI と TCP/IP の対応、および本ノートの守備範囲

注意

CAN は自動車内ネットワークで、TCP/IP のような L3/L4 を標準では持たない。おおむね OSI の物理層+データリンク層に相当する範囲を規定する。「Ethernet と CAN を同じ土俵 (L1/L2) で比べられる」と理解しておくといよい。

1.3 カプセル化：データが旅に出る前の準備

上位層のデータは、下位層を通るたびに**ヘッダ** (必要な**トレーラ**) を付けられていく。これを**カプセル化**という。受信側では逆順に剥がしていく (**デカプセル化**)。各層でのデータの呼び名 (PDU) も決まっている。

例

ブラウザが GET / を送るとき：HTTP メッセージ → (TCP ヘッダ付与) セグメント → (IP ヘッダ付与) パケット → (Ethernet ヘッダ+ FCS 付与) フレーム → 電気/光信号。
受信側はこれを下から順に剥がして HTTP に戻す。

層	データの呼称 (PDU)	付加される代表的な情報
トランスポート	セグメント (TCP) / データグラム (UDP)	ポート番号、シーケンス番号
ネットワーク	パケット	送信元/宛先 IP アドレス
データリンク	フレーム	送信元/宛先 MAC、FCS
物理	ビット列	(信号として送出)

表 1.2 各層での PDU 名と付加情報

まとめ

- ネットワークは層で分けて考える。層間の約束だけが固定。
- OSI 7 層と TCP/IP 4 層の対応を頭に入れておくと迷わない。
- Ethernet と CAN は L1/L2、TCP/IP は L3/L4 が主戦場。
- データは下位層へ行くほどヘッダを付けられる (カプセル化)。

第2章 Ethernet

Ethernet は、有線 LAN で最も広く使われるデータリンク層（+物理層）の規格群で、IEEE 802.3 として標準化されている。本章では「1 本のケーブルに信号を流す話（物理層）」から「誰宛てのフレームかを区別する話（MAC 層）」、そしてスイッチや VLAN まで、レジューメ的に順を追って解説する。

2.1 Ethernet の全体像

定義

Ethernet：ノード間でフレーム（データの塊）を送受信するための、物理層（ケーブル・信号）とデータリンク層（MAC アドレスによる宛先指定）を規定した規格群。OSI の L1/L2 に相当する。

歴史的には 1 本の同軸ケーブルを全員で共有する **バス型** から始まり、**CSMA/CD** で衝突を裁いていた。現在はスイッチで各ノードを 1 対 1 に結ぶ **スター型** + **全二重** が主流で、衝突そのものが起きない。この変遷を知ると「なぜ昔の用語が今は出てこないのか」が腑に落ちる。

2.2 物理層：ケーブルと速度の規格

規格名は <速度>BASE-<媒体> という命名で読める。BASE はベースバンド伝送（変調せずそのまま流す方式）を表す。

規格	速度	媒体	備考
10BASE-T	10 Mbps	ツイストペア (Cat3+)	初期の RJ-45 配線
100BASE-TX	100 Mbps	ツイストペア (Cat5)	Fast Ethernet
1000BASE-T	1 Gbps	ツイストペア (Cat5e)	Gigabit、4 対全使用
10GBASE-T	10 Gbps	ツイストペア (Cat6A)	短距離データセンタ等
1000BASE-SX	1 Gbps	マルチモード光	構内バックボーン
100BASE-T1	100 Mbps	1 対 (非シールド可)	車載 Ethernet
1000BASE-T1	1 Gbps	1 対	車載 Ethernet

表 2.1 代表的な Ethernet 物理規格

要点

車載 Ethernet (100BASE-T1 / 1000BASE-T1) は、軽量化のため **1 対 (2 本) のより線** で全二重を実現するのが特徴。従来 LAN の 100BASE-TX が 2 対、1000BASE-T が 4 対を使うのと対照的で、コネクタやケーブル重量を抑えたい車載要件に合わせて作られている。

2.2.1 オートネゴシエーションと全二重/半二重

リンク両端は **オートネゴシエーション** で、速度と **全二重/半二重** を自動で擦り合わせる。全二重なら送受信を同時に行え、衝突は原理的に起きない。半二重 (古いハブ接続など) では送受信が交互で、衝突管理が必要になる。

2.3 MAC アドレス：誰宛てかを表す 48 ビット

定義

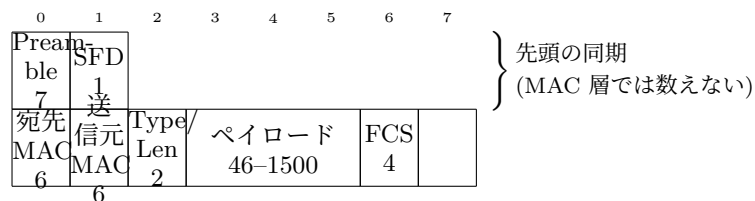
MAC アドレス：ネットワークインタフェース (NIC) に割り当てられた 48 ビット (6 バイト) の識別子。00:1B:44:11:3A:B7 のように 16 進 2 桁をコロン区切りで表記する。上位 24 ビットがベンダ識別子 (OUI)、下位 24 ビットが各社が割り当てる固有番号。

先頭バイトの下位 2 ビットに意味がある点を押さえておくといよい。

- 最下位ビット (I/G ビット)：0 ならユニキャスト、1 なら **マルチキャスト**。全 1 (FF:FF:FF:FF:FF:FF) は **ブロードキャスト**。
- 第 2 ビット (U/L ビット)：0 ならグローバル一意 (工場割り当て)、1 ならローカル管理 (ソフトで付け替えた **ランダム MAC** など)。

2.4 Ethernet フレームの構造

実際に線上を流れるフレームは次の形をしている (Ethernet II / DIX 形式)。数字は **バイト数**。



各フィールドの役割：

Preamble (7B) 10101010 の繰り返し。受信側のクロック同期用。

SFD (1B) 10101011。「ここからフレーム本体」を示す合図。

宛先 MAC (6B) 受け取るべきノードの MAC アドレス。

送信元 MAC (6B) 送り主の MAC アドレス。

Type/Length (2B) 値が 1536(0x0600) 以上なら **EtherType** (上位プロトコル種別。例：0x0800=IPv4, 0x0806=ARP, 0x86DD=IPv6, 0x8100=VLAN タグ)。1500 以下なら長さを表す。

ペイロード (46–1500B)

上位層データ。最大 1500 を **MTU** と呼ぶ。46 未満ならパディングで埋める。

FCS (4B) **CRC-32** による誤り検出符号。壊れたフレームは破棄。

注意

Preamble と SFD は同期用で「MAC フレーム」には含めず、フレーム長にも数えない。最小フレーム長 64 バイト・最大 1518 バイト (VLAN タグなし、宛先 MAC~FCS) という数字はこの本体部分の話である。

また **IFG (Inter-Frame Gap)** として、フレーム間に最低 96 ビット時間の無送信区間を空ける決まりもある。

2.4.1 ジャンボフレームと VLAN タグ

標準 MTU は 1500 だが、データセンタなどでは **ジャンボフレーム** (MTU 9000 程度) で効率を上げることがある。また **IEEE 802.1Q** の VLAN タグ (4 バイト) を宛先 MAC の後ろに挿入すると、1 本の物理線を論理的に複数の LAN へ分割できる。タグには VLAN ID (12 ビット、最大 4094 個) と優先度 (PCP, 3 ビット) が入る。

2.5 かつての主役：CSMA/CD

半二重・共有媒体の時代、複数ノードが同時に送ると信号がぶつかった (**衝突**)。これを裁く方式が **CSMA/CD** である。

定義

CSMA/CD (Carrier Sense Multiple Access with Collision Detection) :

1. **CS** : 送る前に線が空いているか確認 (キャリアセンス)。
2. **MA** : 空いていれば送信開始 (多重アクセス)。
3. **CD** : 送信中も監視し、衝突を検出したら直ちに中断。**ジャム信号**を流して全員に衝突を知らせ、**ランダムな待ち時間** (指数バックオフ) の後に再送する。

要点

現代のスイッチ+全二重環境では各リンクが 1 対 1 なので**衝突は起きず、CSMA/CD は無効化されている**。だが「最小フレーム長 64 バイト」のような現行仕様は、衝突を確実に検出するための制約に由来する。歴史を知ると数字の理由が分かる、という典型例。

2.6 ハブとスイッチ：衝突ドメインとブロードキャストドメイン

リピータハブ 受けた信号を全ポートへそのまま中継するだけ (L1)。全ポートが 1 つの**衝突ドメイン**になり、規模を上げると衝突が増える。

スイッチ 宛先 MAC を見て該当ポートだけへ転送する (L2)。**MAC アドレステーブル**を学習し、ポートごとに衝突ドメインを分離する。ただしブロードキャストは全ポートへ流れる。

定義

衝突ドメイン : 同時に送ると衝突しうるノードの範囲。スイッチの各ポートで分割される。

ブロードキャストドメイン : ブロードキャストフレームが届く範囲。スイッチでは分割されず、ルータまたは **VLAN** で初めて分割される。

2.6.1 スwitchの MAC 学習 (要点)

1. フレーム受信時、その**送信元 MAC** と**受信ポート**の対応を表に記録。
2. 宛先 MAC が表にあれば、そのポートだけへ転送 (**フォワーディング**)。
3. 表になければ受信ポート以外の全ポートへ転送 (**フラッディング**)。
4. 一定時間使われない項目は破棄 (**エージング**)。

例

`ip link / ip neigh` (Linux) や `arp -a` (Windows) で、自ホストが学習した MAC と IP の対応を確認できる。スイッチ側のテーブルは管理画面や `show mac address-table` 等で見える。

まとめ

- Ethernet は L1/L2。速度 BASE 媒体で物理規格を読む。車載は 1 対線の 100/1000BASE-T1。
- フレームは宛先/送信元 MAC + EtherType + ペイロード + FCS(CRC-32)。最小 64 ・ 最大 1518 バイト、MTU 1500。
- MAC は 48 ビット。先頭バイトの I/G ・ U/L ビットに意味がある。
- 現代はスイッチ+全二重で衝突なし。CSMA/CD は歴史的背景として理解。
- 衝突ドメインはスイッチで分割、ブロードキャストドメインはルータ/VLAN で分割。

第 3 章 TCP/IP

TCP/IP は、世界中の異なるネットワークを相互接続するためのプロトコル群で、インターネットの土台である。本章では L3 の **IP**（住所と配送）と、L4 の **TCP / UDP**（アプリ同士の通信路）を中心に解説する。Ethernet 章が「同じ LAN 内の話」だったのに対し、ここは「LAN を越えて世界中へ届ける話」になる。

3.1 IP：エンドツーエンドの住所と配送

定義

IP (Internet Protocol)：送信元から宛先まで、複数のネットワークをルータで中継しながらパケットを配送する L3 プロトコル。**ベストエフォート**（到達・順序・重複なしを保証しない）かつ**コネクションレス**（事前の接続確立なし）。信頼性は上位の TCP が担う。

3.1.1 IPv4 アドレスとサブネット

IPv4 アドレスは 32 ビットを 8 ビットずつ 192.168.1.10 のように表記する。アドレスは **ネットワーク部**と**ホスト部**に分かれ、その境界を **サブネットマスク**（または **CIDR** の /24 表記）で示す。

例

192.168.1.10/24 の場合：

- サブネットマスク 255.255.255.0（上位 24 ビットがネットワーク部）
- ネットワークアドレス：192.168.1.0（ホスト部が全 0）
- ブroadcastアドレス：192.168.1.255（ホスト部が全 1）
- 割当可能ホスト：192.168.1.1～192.168.1.254（254 個）

要点

プライベートアドレス（10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16）は組織内専用で、そのままではインターネットに出られない。出る際は **NAT**（次節）でグローバルアドレスに変換される。

3.1.2 IPv4 ヘッダ

0	4	8	16	19	31
Ver	IHL	ToS/DSCP	Total Length		
Identification			Flags	Fragment Offset	
TTL		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options（任意）+ Padding					

押さえるべきフィールド：

Version 4（IPv4）。

IHL ヘッダ長（4 バイト単位）。オプション無しで 5 = 20 バイト。

Total Length ヘッダ＋データの総バイト数（最大 65535）。

Identification/Flags/Fragment Offset

フラグメント（分割）制御。MTU を超えるパケットを分割・再構成するための情報。

TTL **Time To Live**。ルータを 1 つ越えるごとに 1 減り、0 で破棄。無限ループ防止。traceroute はこれを利用して経路を可視化する。

Protocol 上位プロトコル番号（6=TCP, 17=UDP, 1=ICMP）。

Header Checksum

ヘッダのみの誤り検出（データは対象外）。

Source/Destination IP

送信元・宛先アドレス。

3.1.3 IPv6 のひとこと

IPv4 アドレス枯渇への対応が **IPv6** (128 ビット、2001:db8::1 のような表記)。ヘッダを固定長で簡素化し、チェックサムを廃止 (上位層に委ねる)、フラグメントは原則送信元のみが行う、など処理を軽くしている。

3.2 ARP と ICMP : IP を支える縁の下

ARP **Address Resolution Protocol**。同一 LAN 内で「この IP の MAC は？」を問い合わせる。宛先 IP に対する MAC をブロードキャストで尋ね、本人がユニキャストで答える。結果は **ARP キャッシュ** に保持。IP → MAC の橋渡し役。

ICMP **Internet Control Message Protocol**。到達不能・TTL 超過などのエラー通知や、ping (Echo Request/Reply) に使う診断用。

例

ping 8.8.8.8 は ICMP Echo を投げて応答時間を測る。
tracert (Win は tracert) は TTL を 1,2,3... と増やして送り、各ルータからの「TTL 超過」ICMP を集めて経路を割り出す。

3.3 TCP : 信頼性のある通信路

定義

TCP (Transmission Control Protocol) : コネクション型・信頼性ありの L4 プロトコル。順序保証・再送による欠落補償・フロー制御・輻輳制御を提供し、バイトストリームとしてデータを届ける。HTTP, SSH, メールなど「確実に届けたい」通信で使う。

3.3.1 TCP ヘッダ

0	4	10	16	32
Source Port			Destination Port	
Sequence Number				
Acknowledgment Number				
Data Off	Rsvd	Flags	Window Size	
Checksum			Urgent Pointer	
Options（任意）+ Padding				

Source/Dest Port 16 ビットのポート番号。どのアプリ宛てかを区別。

Sequence Number 送るデータの先頭バイト位置 (番号)。順序管理の要。

Acknowledgment Number

「次に欲しいバイト番号」。ここまで受領済みを表す。

Data Offset

ヘッダ長 (オプション込み)。

Flags

制御ビット。**SYN** (接続開始), **ACK** (確認応答), **FIN** (切断要求), **RST** (強制リセット), **PSH** (即時引き渡し), **URG** (緊急)。

Window Size

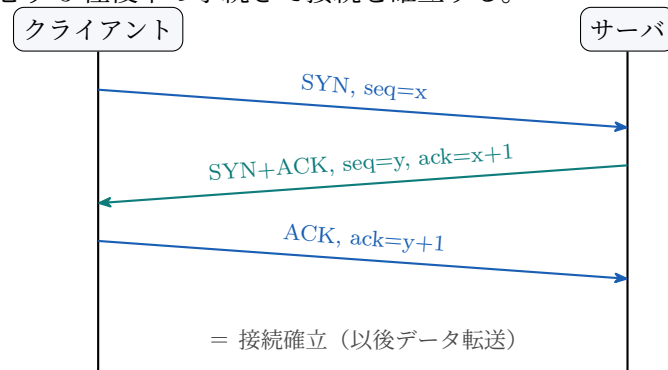
受信側がまだ受け取れるバイト数。**フロー制御**に使う。

Checksum

ヘッダ+データ+擬似ヘッダの誤り検出。

3.3.2 3ウェイハンドシェイク (接続確立)

TCP は通信前に必ず 3 往復半の手続きで接続を確立する。



1. クライアント→サーバ: **SYN** (自分の初期シーケンス番号 x を通知)。

2. サーバ→クライアント：**SYN+ACK**（自分の番号 $y + x + 1$ を確認）。
3. クライアント→サーバ：**ACK** ($y + 1$ を確認)。これで双方向に確立。

注意

切断は **FIN** を双方が送り合う **4ウェイ** が基本（半二重ずつ閉じる）。また、確立済みでない/不正なパケットには **RST** が返り、接続が即座にリセットされる。パケットキャプチャで RST を見たら「拒否された/途切れた」の合図。

3.3.3 信頼性のしくみ（要点）

要点

再送	ACK が来ない／重複 ACK が続くと、データを再送する。
順序整理	シーケンス番号で並べ替え、欠落を検出する。
フロー制御	受信側の Window Size （受信バッファ空き）を超えて送らない。受け手を溢れさせない仕組み。
輻輳制御	ネットワークの混雑を推測し送信レートを調整（スロースタート→輻輳回避など）。経路を溢れさせない仕組み。

フロー制御は「受信ホストを守る」、輻輳制御は「経路（網）を守る」と区別すると整理できる。

3.4 UDP：軽量・コネクションレス

定義

UDP (User Datagram Protocol)：コネクションレスで再送も順序保証もしない最小限の L4 プロトコル。ヘッダはわずか 8 バイト（送信元/宛先ポート、長さ、チェックサム）。オーバーヘッドが小さく低遅延。

DNS、DHCP、NTP、音声/動画のリアルタイム配信、QUIC の土台など、「多少落としても速さ・即時性が大事」な用途に向く。信頼性が要るならアプリ側で工夫する。

観点	TCP	UDP
接続	コネクション型（要ハンドシェイク）	コネクションレス
信頼性	再送・順序保証あり	なし（ベストエフォート）
ヘッダ	20 バイト～	8 バイト固定
速度/遅延	制御が入る分やや重い	軽量・低遅延
主な用途	HTTP, SSH, メール	DNS, DHCP, 動画/音声, ゲーム

表 3.1 TCP と UDP の比較

3.5 ポート番号と NAT

ポート番号（16 ビット）は同一ホスト内のどのアプリ宛てかを区別する。0--1023 は **ウェルノウンポート**（HTTP 80, HTTPS 443, SSH 22, DNS 53 など）。**IP アドレス+ポート番号**の組を**ソケット**と呼び、通信の端点を一意に表す。

定義

NAT（Network Address Translation）：プライベート IP とグローバル IP を変換する仕組み。家庭/企業では **NAPT（IP マスカレード）**が一般的で、「グローバル IP 1 個 + ポート番号の付け替え」で多数の端末が同時に外へ出られる。

まとめ

- IP は L3。ベストエフォートで世界中へ配送。TTL でループ防止、Protocol 番号で上位を示す。サブネット/CIDR で住所を区切る。
- ARP は IP → MAC、ICMP は診断/エラー通知（ping, traceroute）。
- TCP は信頼性あり：3 ウェイで確立、seq/ack で順序と再送、Window でフロー制御、別途輻輳制御。
- UDP は軽量・コネクションレス。即時性重視の用途向け。
- ポートでアプリを区別し、IP + ポート = ソケット。外へ出る時は NAT。

第4章 CAN (Controller Area Network)

CAN は、自動車の ECU 同士をつなぐために Bosch 社が開発した車載ネットワークで、ISO 11898 として標準化されている。Ethernet/TCP/IP が「大量データを世界へ」だったのに対し、CAN は「短いメッセージを、確実に・リアルタイムに・少ない配線で」やり取りすることに特化している。本章は L1/L2、すなわち物理層とデータリンク層が中心になる。

4.1 CAN の設計思想

要点

CAN の要点は次の 4 つ。これが他のネットワークと毛色が違う理由。

- **メッセージベース**：宛先アドレスではなく、メッセージ自体に付いた **ID** で内容を識別する。送信は基本ブロードキャストで、各 ECU は受け取りたい ID だけを**フィルタ**で拾う。
- **マルチマスタ**：どの ECU も自発的に送信を始められる。
- **優先度付きアービトラージョン**：同時送信時は ID の小さい（優先度の高い）メッセージが、衝突で壊れずに勝ち残る。
- **堅牢性**：差動信号＋強力な誤り検出で、ノイズの多い車内でも信頼性が高い。

4.2 物理層：差動信号とドミナント/レセシブ

CAN は **CAN_H** と **CAN_L** の 2 本のより線による**差動伝送**を使う。両端を **120 Ω** の終端抵抗で終端したバス型トポロジである。

定義

ビットは 2 状態で表される：

- **ドミナント (dominant, 論理 0)**：CAN_H と CAN_L に電位差を作る（能動的に駆動）。
- **レセシブ (recessive, 論理 1)**：両線がほぼ同電位（駆動しない）。

重要な性質：ドミナントとレセシブが同時に出されると、バスはドミナントになる。これが「ワイヤード AND」的な振る舞いで、後述のアービトレーションの鍵。

注意

差動なので、両線に等しく乗るコモンモードノイズは受信側の差分で打ち消される。これが車内の電氣的に厳しい環境で CAN が強い理由。終端抵抗 120Ω をバス両端に正しく入れることも信号品質に必須（合成 60Ω ）。片方欠けると反射でエラーが増える。

4.3 CAN フレームの構造（標準フォーマット）

最もよく使うデータフレーム（標準 11 ビット ID）の構造。

各フィールドを左から順に並べると次のとおり（各ボックス下の数字がビット数。幅は見やすさ優先で等幅にしてある）。

SOF ₁	Ident ₁₁	RTR ₁	IDE ₁	r0 ₁	DLC ₄	Data ₀₋₆₄	CRC ₁₅	CRC _{del} ₁	ACK _{slot} ₁	ACK _{del} ₁	EOF ₇
------------------	---------------------	------------------	------------------	-----------------	------------------	----------------------	-------------------	---------------------------------	----------------------------------	---------------------------------	------------------

このうち先頭の SOF~Identifier~RTR がアービトレーション領域で、ここで送信権（優先度）が決まる（4.4 節）。

SOF	Start of Frame. 1 ドミナントビットでフレーム開始を知らせ、全ノードの同期をとる。
Identifier	11 ビットの ID。メッセージの種類を表すと同時に 優先度 にもなる（小さいほど高優先）。
RTR	Remote Transmission Request. 0 = データフレーム、1 = リモートフレーム（データ要求）。
IDE	0 = 標準フォーマット（11 ビット）、1 = 拡張フォーマット（29 ビット）。
DLC	Data Length Code. データ部のバイト数（0~8）。
Data	実データ。 最大 8 バイト （クラシック CAN）。
CRC (15 bit)	巡回冗長検査による誤り検出。続く CRC デリミタは 1 レセシブ。
ACK slot	送信側はレセシブで送る。 正しく受信できたノードがドミナントで上書きして「受け取った」と知らせる。
EOF	End of Frame. 7 レセシブビットでフレーム終了。

例

拡張フォーマット（CAN 2.0B）は ID が 29 ビット（11 ビット + SRR/IDE を挟んで 18 ビット拡張）。識別子の数を増やせるが、1 フレームが長くなる。多くの車載では標準 11 ビットと併用される。

4.4 アービトレーション：衝突しても壊れない調停

CAN の真骨頂が、複数ノードが同時送信したときの**非破壊アービトレーション**である。Ethernet の CSMA/CD が「衝突→中断→再送」だったのに対し、CAN は **CSMA/CA**（Collision Avoidance）で**勝者のフレームを壊さない**。

定義

各ノードは ID を 1 ビットずつ送りながら、同時に**バスの状態を読む**。

- 自分がレセシブ (1) を出したのにバスがドミナント (0) だった ⇒ 他により優先度の高い送信者がいる。**自分は送信を降る**。
- 最後まで「自分の出力＝バスの状態」だったノードが**勝者**となり、そのままデータ送信を続ける。

ドミナント = 0 が優先なので、**ID の数値が小さいほど高優先**になる。

例

ID = 0x100 (…0100…) と ID = 0x101 (…0101) が同時送信した場合、上位ビットは一致し、最後のビットで 0x100 がドミナント (0)、0x101 がレセシブ (1) を出す。バスはドミナントになるので 0x101 側が降り、0x100 が勝つ。**衝突したのに、勝者のフレームは 1 ビットも壊れていない点がミソ**。

要点

この仕組みのおかげで、ブレーキのような高優先メッセージは「小さい ID」を割り当てておけば、混雑時でも遅延なく通る。**ID 設計＝優先度設計**であり、車載ネットワーク設計の肝になる。

4.5 ビットスタッフィングと同期

CAN には別クロック線が無く、各ノードは受信信号のエッジで同期する（NRZ 符号）。同じ値が続くとエッジが現れず同期がずれるため、ビットスタッフィングを行う。

定義

ビットスタッフィング：同一極性のビットが 5 個連続したら、送信側が反対極性のビットを 1 個挿入する。受信側はそれを取り除く。これで定期的にエッジが生じ、再同期できる。

注意

スタップビットは SOF～CRC まだが対象。これによりフレーム長はデータ内容で変動する。意図的にスタップルール違反を起こしてエラーを通知するのが次節のエラーフレーム。

4.5.1 ビットタイミング（参考）

1 ビット時間は 同期セグメント／伝播セグメント／位相バッファ 1・2 に分割され、サンプルポイント（バス値を読む瞬間）の位置を調整できる。ずれは SJW（再同期ジャンプ幅）の範囲で補正される。ボーレートは高速 CAN で最大 1 Mbps（クラシック）。

4.6 エラー検出とフォールトコンフィニメント

CAN は 5 種類の誤り検出を多重に持つ。1 つでも引っかかればエラーフレームを流して全ノードに知らせ、送信者は再送する。

誤り検出	内容
ビットエラー	送信した値とバス上の値が（アービト/ACK 以外で）食い違う
スタップエラー	6 ビット以上同一極性が続いた（スタップ規則違反）
CRC エラー	受信 CRC が計算値と不一致
フォーマットエラー	固定フォーマット部（デリミタ等）の値が異常
ACK エラー	誰も ACK スロットをドミナントにしなかった

表 4.1 CAN の 5 つの誤り検出

4.6.1 エラーカウンタとバスオフ

定義

各ノードは **送信エラーカウンタ (TEC)** と **受信エラーカウンタ (REC)** を持ち、エラーで増え、正常送受信で減る。値に応じて状態が遷移する。

エラーアクティブ 通常状態。エラー検出時は **アクティブエラーフレーム** (ドミナント) で積極的に通知する。

エラーパッシブ (TEC または REC > 127)
故障気味。控えめな **パッシブエラーフレーム** しか出せず、連続送信時に待ち時間が増える。

バスオフ (TEC > 255) 自分が故障源とみなし、**バスから切り離れる**。他ノードの通信を妨げないための隔離。

要点

この **フォールトコンフィンメント** (故障の封じ込め) により、壊れた 1 ノードがバス全体を巻き込んで通信不能にすることを防ぐ。「故障したノードは自ら黙る」設計が、安全が問われる車載で重視される所以。

4.7 CAN FD：データを増やし高速化

定義

CAN FD (Flexible Data-rate)：クラシック CAN の拡張で、

- データ部を **最大 64 バイト** まで拡張 (8 → 64)。
- **データ部だけビットレートを上げる** (アービト部は従来速度のまま、データ部は数 Mbps)。アービトレーションの仕組みは保ちつつ高速化。
- CRC を強化 (長いデータに対応)。

カメラ前処理データや診断など、クラシック CAN の 8 バイトでは足りない用途に有効。

4.8 Ethernet との棲み分け（まとめにかえて）

近年は**車載 Ethernet**（第 2 章の 100/1000BASE-T1）と CAN が併存する。大容量（カメラ・自動運転センサ）は Ethernet、制御系の確実・低遅延な短メッセージは CAN/CAN FD、という役割分担が一般的である。

観点	CAN / CAN FD	車載 Ethernet
1 フレームのデータ	8 / 最大 64 バイト	最大 1500 バイト
速度	～1 Mbps / 数 Mbps	100 Mbps～数 Gbps
調停	ID 優先度で非破壊調停	スイッチ+全二重
向く用途	制御系の確実・低遅延	大容量センサ/映像

表 4.2 CAN と車載 Ethernet の棲み分け

まとめ

- CAN はメッセージベース・マルチマスタの車載ネットワーク（L1/L2）。
- 物理層は CAN_H/CAN_L の差動、ドミナント (0)/レセシブ (1)、終端 120 Ω。
- アービトレーションは非破壊：ID が小さいほど高優先、勝者は壊れない。**ID 設計＝優先度設計**。
- ビットスタッフィングで同期、CRC ほかに 5 種の誤り検出。
- エラーカウンタで**エラーアクティブ→パッシブ→バスオフ**と隔離（フォールトコンフィニメント）。
- CAN FD は最大 64 バイト+データ部高速化。大容量は車載 Ethernet と分担。

第5章 演習：パケットを解読する

ここまでで「フレーム/パケットはフィールドの並びである」ことを見てきた。本章では、Wireshark で実際に取得した 1 パケット分のバイト列を、仕様表（プロトコル定義）に従って人手で解読する演習を行う。キャプチャしたデータが「ただの 16 進の羅列」から「意味のある数値」へ変わる感覚をつかむのが狙いである。

要点

この演習で身につく読み方：

- ・エンディアン：複数バイトの整数を、どちらのバイトから読むか。
- ・符号の有無：同じビット列でも uint と int で意味が変わる。
- ・スケール／オフセット：生値（raw）に分解能を掛けて物理量へ直す。
- ・ASCII：バイトを文字として読む。
- ・予約領域：すべてのバイトが意味を持つわけではない。

5.1 問題

ある車載機器が UDP で送信するテレメトリを Wireshark で取得したところ、UDP ペイロード（Ethernet/IP/UDP ヘッダを剥がした後のアプリ層データ）は 37 バイトで、次の 16 進バイト列であった。

```
26 00 25 00 03 00 00 00 11 ff f0 fe 99 99 99 99
99 99 99 99 99 52 52 99 99 99 99 99 00 ff 70 4b
03 4b 03 00 00
```

バイト位置（オフセット）を付けて並べ直すと次のとおり。バイト位置 = 行オフセット + 列の +n（例：0x08 行の +2 は 17 バイト目 = 位置 10）。

オフセット	+0	+1	+2	+3	+4	+5	+6	+7
0x00	26	00	25	00	03	00	00	00
0x08	11	ff	f0	fe	99	99	99	99
0x10	99	99	99	99	99	52	52	99
0x18	99	99	99	99	00	ff	70	4b
0x20	03	4b	03	00	00			

5.1.1 与えられた仕様（プロトコル定義）

このペイロードのフォーマットは下表で定義されている。多バイト整数はすべてリトルエンディアン、位置は 0 始まり。これを使って各 Variable の値を求めるのが本問である。

データ型表記：u8/u16/u32 = 符号なし整数（8/16/32 ビット）、i16 = 符号付き 16 ビット、bool = 真偽値（1 バイト）、char[N] = ASCII 文字 N 個、u8[N] = バイト配列。手計算の手順は付録 A を参照（16 進 → 10 進、エンディアン、符号付き/なしの計算）。

#	Variable	Byte Position	Data type	単位・スケール（備考）
1	msg_id	0–1	u16	プロトコル識別子
2	length	2–3	u16	パケット全長 [byte]
3	seq	4–7	u32	連番カウンタ
4	msg_type	8	u8	メッセージ種別
5	status_flags	9	u8	8 ビットのフラグ（後述）
6	accel_x	10–11	i16	$\times 0.01$ [m/s ²]（符号付き）
7	reserved1	12–20	u8[9]	予約（未使用）
8	unit_id	21–22	char[2]	ASCII 文字
9	reserved2	23–27	u8[5]	予約（未使用）
10	gnss_fix	28	bool	GNSS 測位確定（0=false）
11	signal_quality	29	u8	0–255
12	coolant_temp	30	u8	値 – 40 [°C]（オフセット）
13	speed	31–32	u16	$\times 0.1$ [km/h]
14	target_speed	33–34	u16	$\times 0.1$ [km/h]
15	crc16	35–36	u16	チェックサム

5.1.2 設問

1. [基本] 各 Variable の生値（raw）を求めよ。とくに多バイト整数はリトルエンディアンで読むこと。
2. [変換] スケール・オフセットのある accel_x, coolant_temp, speed, target_speed を物理量に直せ。accel_x は符号付きである点に注意。
3. [整合性] length の値は、実際のパケット長（37 バイト）と一致するか確認せよ。
4. [ビット] status_flags を 8 桁の 2 進数で表せ。ビット定義が下表のとき、どの状態が ON か答えよ。

bit	意味	bit	意味
0	イグニッション ON	4	右ウィンカー
1	ブレーキ踏込	5	ヘッドライト
2	アクセル踏込	6	ワイパー
3	左ウィンカー	7	システム正常

5. [発展・プラス] 次に答えよ。

- (a) reserved1/2 を除く「意味のある」バイト数は何バイトか。
- (b) もしこのバイト列をビッグエンディアンと誤読すると、msg_id はいくつになるか。なぜ最初にエンディアンを確かめるべきかを、length を使って説明せよ。
- (c) crc16 が 0x0000 であることは何を示唆するか。

注意

リトルエンディアンでは「先に来たバイトが下位」。例えば 26 00 は $0x00 \times 256 + 0x26 = 0x0026 = 38$ であって、 $0x2600 = 9728$ ではない。ここを取り違えると以降すべてズレるので、最初に必ず確認する。(並べ替え・重み和の手順は付録 A.3、16 進 → 10 進は付録 A.2 を参照。)

5.2 解答・解説

5.2.1 設問 1・2：各フィールドの値

リトルエンディアンで読み、スケール／オフセットを適用すると次のとおり。各計算の手順は付録 A（16 進 → 10 進 A.2、エンディアン A.3、符号付き A.5）にまとめてある。

Variable	該当バイト	生値 (raw)	物理量	読み方
msg_id	26 00	38	—	0x0026
length	25 00	37	37 byte	0x0025。全長と一致
seq	03 00 00 00	3	—	0x00000003
msg_type	11	17	—	0x11
status_flags	ff	255	—	1111 1111
accel_x	f0 fe	−272	−2.72 m/s ²	0xFEFO = 符号付き
reserved1	99×9	—	—	予約（無視）
unit_id	52 52	—	“RR”	ASCII
reserved2	99×5	—	—	予約（無視）
gnss_fix	00	0	false	0=false
signal_quality	ff	255	255/255	最良
coolant_temp	70	112	72°C	112 − 40
speed	4b 03	843	84.3 km/h	0x034B × 0.1
target_speed	4b 03	843	84.3 km/h	0x034B × 0.1
crc16	00 00	0	—	0x0000

とくに引っかかりやすい 3 つを補足する。

例

符号付き accel_x (f0 fe)：LE で読むと 0xFEFO = 65264。最上位ビットが 1 ($\geq 0x8000$) なので**負数**。16 ビット 2 の補数では $65264 - 65536 = -272$ 。スケール 0.01 を掛けて **−2.72 m/s²**（減速＝後ろ向きの加速度）。（2 の補数の手順は付録 A.5。）

例

ASCII unit_id (52 52)：0x52 は ASCII の 'R'。よって "RR"。数値ではなく**文字**として解釈するフィールドである点に注意。

例

オフセット `coolant_temp` (70) : $0x70 = 112$ 。仕様は「値 -40 」なので $112 - 40 = 72$ °C。
負の温度も 1 バイトで表せるよう、 -40 を 0 点にずらす常套手段。

5.2.2 設問 3 : `length` の整合性

`length = 0x0025 = 37` で、実際のペイロード長 37 バイトと一致する。自己記述的な長さフィールドは、受信側が「全部受け取れたか」を確かめる重要な手がかり。ここがズレていたらエンディアンを読み違いかパケットの途中欠けを疑う。

5.2.3 設問 4 : `status_flags` のビット展開

$0xFF = 11111111$ (bit0~bit7 がすべて 1)。したがって全状態が ON : イグニッション ON・ブレーキ踏込・アクセル踏込・左右ウィンカー・ヘッドライト・ワイパー・システム正常。(実車ならアクセルとブレーキ同時 ON など不自然だが、本問は全ビット立ったテストパターンである。)

要点

1 バイトの $0xFF$ は「255 という数値」と読むこともできるが、フラグとして定義されていれば「8 個のスイッチが全部 ON」を意味する。同じビット列でも、仕様がどう定義しているかで意味が決まる。

5.2.4 設問 5 : 発展 (プラス)

- (a) 予約は `reserved1(9) + reserved2(5) = 14` バイト。意味のあるバイトは $37 - 14 = 23$ バイト。「キャプチャ長=有効データ量」ではないことが分かる。
- (b) ビッグエンディアンと誤読すると `msg_id` は $0x2600 = 9728$ 。同様に `length` は $0x2500 = 9472$ となり、実際の 37 バイトと大きく矛盾する。この矛盾こそが「エンディアンを取り違えている」という危険信号。既知の値を持つフィールド (`length` など) でエンディアンを最初に検算するのが定石。
- (c) `crc16 = 0x0000` は、送信側が CRC を未計算 (ブレースホルダ) のまま送っている、あるいはテスト送出である可能性を示唆する。本来は全フィールドから計算した値が入る。

まとめ

- 16 進ダンプはバイト位置 × 仕様表で初めて意味を持つ。
- 多バイト整数はエンディアンを最初に確定（既知値で検算）。
- uint/int の別、スケール・オフセット、ASCII、ビットフラグ、予約領域を仕様どおりに適用する。
- length や crc のような整合性フィールドは、読み違い・欠損の検出に使える。

付録 A 手計算リファレンス：16 進数の読み方

第 A 章は、演習（第 5 章）を電卓やコードに頼らず手で解くための計算手順集である。16 進ダンプを見て頭の中で値に変換できると、キャプチャ解析やデバッグの速度が段違いに上がる。

A.1 下準備：16 進 1 桁の対応表

まずこの表を覚える（または手元に置く）。16 進 1 桁は **4 ビット** にちょうど対応する。

16 進	10 進	2 進	16 進	10 進	2 進
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	A	10	1010
3	3	0011	B	11	1011
4	4	0100	C	12	1100
5	5	0101	D	13	1101
6	6	0110	E	14	1110
7	7	0111	F	15	1111

A.2 16 進数 → 10 進数

定義

各桁に**桁の重み** 16^n （右端 = $16^0 = 1$ ）を掛けて足す。

$$0xd_k \cdots d_1 d_0 = \sum_{n=0}^k d_n \times 16^n \quad (16^0=1, 16^1=16, 16^2=256, 16^3=4096)$$

例

- $0x25 = 2 \times 16 + 5 = 32 + 5 = \mathbf{37}$
- $0x4B = 4 \times 16 + 11 = 64 + 11 = \mathbf{75}$ (B= 11 は前表より)
- $0x034B = 3 \times 256 + 4 \times 16 + 11 = 768 + 64 + 11 = \mathbf{843}$
- $0xFEFO = 15 \times 4096 + 14 \times 256 + 15 \times 16 + 0 = 61440 + 3584 + 240 = \mathbf{65264}$

要点

2 桁 (1 バイト) なら「上の桁 $\times 16$ + 下の桁」だけで暗算できる。例: $0x70 = 7 \times 16 + 0 = 112$ 、 $0x11 = 1 \times 16 + 1 = 17$ 。

A.3 複数バイトの並べ替え：エンディアン

複数バイトの整数は、メモリ上のバイトの並び順を先に確定しないと値が出せない。

定義

リトルエンディアン (LE)：先に来たバイトが下位 (小さい桁)。

ビッグエンディアン (BE)：先に来たバイトが上位 (大きい桁)。

手順 A：並べ替えてから変換

バイトを逆順に書き直して 1 つの 16 進数にし、A.2 節で 10 進へ。

例

LE のバイト列 4b 03 を読む：逆順に並べて 03 4b $\rightarrow 0x034B = 843$ 。

(同じ 4b 03 を BE と誤読すると $0x4B03 = 19203$ で大違い。)

手順 B：重み 256 の和 (並べ替え不要)

i 番目のバイト (先頭が $i = 0$) に 256^i を掛けて足す。LE ならこれが速い。

$$\text{値}_{\text{LE}} = \sum_i \text{byte}_i \times 256^i \quad (256^0=1, 256^1=256, 256^2=65536, \dots)$$

例

- 26 00 (LE) $= 0x26 \times 1 + 0x00 \times 256 = 38 + 0 = 38$
- 03 00 00 00 (LE) $= 3 \times 1 + 0 + 0 + 0 = 3$
- 4b 03 (LE) $= 75 \times 1 + 3 \times 256 = 75 + 768 = 843$

注意

エンディアンは最初に確かめる。既知の値を持つフィールド（例：演習の `length`）で検算し、LE で読んだ値が筋の通る数になる方を採用する。ここを間違えると以降のフィールドが全部ずれる。

A.4 unsigned int（符号なし整数）

符号なしはそのまま大きさ。ビット幅で表せる範囲だけ決まっている。

型	ビット幅	範囲
u8	8	0 ~ 255
u16	16	0 ~ 65535
u32	32	0 ~ 4,294,967,295

例

u16 の `speed = 4b 03` (LE) = 843。スケール 0.1 を掛けて $843 \times 0.1 = 84.3$ km/h。 (値の出し方は A.3 節の手順 B。)

A.5 signed int（符号付き整数・2 の補数）

符号付きは **2 の補数** で表される。最上位ビット (MSB) が符号 (1 なら負)。

定義

N ビットの符号付き整数 iN の読み方：

1. まず符号なしとして値 U を求める (A.4 節)。
2. MSB を見る。 $U < 2^{N-1}$ なら正で、値 = U 。
3. $U \geq 2^{N-1}$ (MSB=1) なら負で、値 = $U - 2^N$ 。

範囲は $-2^{N-1} \sim 2^{N-1} - 1$ (i16 なら $-32768 \sim 32767$)。

例

i16 の `accel_x = f0 fe` (LE) を読む：

1. 符号なし： $0xFEFO = 65264$ 。

2. 上位バイト $0xFE = 1111\ 1110$ 、MSB=1 なので**負**。 $(65264 \geq 2^{15} = 32768)$ でも判定できる。)

3. 値 $= 65264 - 2^{16} = 65264 - 65536 = -272$ 。

スケール 0.01 を掛けて $-272 \times 0.01 = -2.72\text{ m/s}^2$ 。

要点

別法 (大きさを直接出す): 負数の大きさ $= 2^N - U$ 。 $0xFEFO$ なら $65536 - 65264 = 272$ なので -272 。ビット反転して $+1$ ($0xFEFO \rightarrow 0x010F = 271$, $+1 = 272$) でも同じ。

A.6 bool (真偽値)

定義

bool は通常 1 バイトで表し、 $0x00 = \text{false}$ 、それ以外 $= \text{true}$ (慣習的に $0x01$ を true に使う)。

例

演習の `gnss_fix` (byte 28) $= 0x00 \Rightarrow \text{false}$ 。

A.7 ビットフラグの展開

1 バイトを 8 個のスイッチとみなす場合は、2 進数に展開して各ビットを読む。16 進 1 桁 $= 4$ ビットなので、上位ニブル・下位ニブルを A.1 節の表で引くだけでよい。

例

- $0xFF \rightarrow 1111\ 1111$ (全ビット ON)
- $0x11 \rightarrow 0001\ 0001$ (bit0 と bit4 が ON)

ビット番号は右端 (最下位) が bit0。 k 番目のビットは $(\text{byte} \gg k) \& 1$ で取り出せる。

まとめ

- 16 進 $\rightarrow$ 10 進：桁に 16^n を掛けて和（2 桁は「上 $\times 16$ + 下」）。
- 多バイトは**エンディアンを先に確定**。LE は「先のバイトが下位」。
- u8/u16/u32 は大きさそのまま。i16 等は **2 の補数**（MSB=1 なら $U - 2^N$ ）。
- bool は 1 バイト（0=false）。フラグは 2 進展開して各ビットを読む。